

PressEase: Comprehensive Project Documentation

1. Project Overview

PressEase is a full-stack web application designed for managing laundry and press shop operations. It consists of a Node.js/Express backend that handles API requests, business logic, and database interactions, and a Next.js (React) frontend that provides a modern, responsive user interface.

2. Architecture & Connections

The application follows a standard Client-Server architecture:

- Frontend (Client):** Built with Next.js, it communicates with the backend via RESTful APIs using standard HTTP methods (GET, POST, PUT, DELETE). The `lib/api.ts` file acts as the central hub for all outward network requests, carrying an authorization token.
- Backend (Server):** Built with Node.js and Express. It receives requests, validates authentication via JWT (JSON Web Tokens), processes business logic in controllers, interacts with a MongoDB database via Mongoose models, and returns JSON responses.

Data Flow Example (Creating an Order):

- User Action:** The user fills out a "New Order" form on the frontend and clicks submit.
- Frontend API Call:** `frontend/src/lib/api.ts` sends a POST request with the order data and the user's JWT token to the backend.
- Backend Route:** `backend/src/routes/orders.js` receives the POST request.
- Middleware Validation:** `backend/src/middleware/auth.js` verifies the JWT token. If valid, it passes control to the next function.
- Controller Logic:** `backend/src/controllers/orderController.js` processes the data. It calculates costs and creates a new `Order` document.
- Database Save:** The Mongoose `Order` model (`backend/src/models/Order.js`) saves it to MongoDB.
- Utility Triggers:** The controller might trigger `pdfGenerator.js` to create a bill and `whatsapp.js` to notify the customer.
- Response:** The backend sends a success response back to the frontend, which then updates the UI.

3. Backend Components Detailed Breakdown

Entry Point

- `src/server.js`: The main entry point. It connects to the MongoDB database, configures Express middlewares (like CORS, JSON body parser), and registers all the route handlers (e.g., `/api/auth`, `/api/orders`). It starts the server listening on a specified port.

Models (`src/models/`)

These files define the structure of your MongoDB database collections using Mongoose schemas.

- `User.js`: Stores shop owners/admins. Contains username, password (hashed), and roles. Used for authentication.
- `Customer.js`: Stores customer details (name, phone, address). Connected to orders via a relational ID.
- `Order.js`: The core operational model. Stores order details, including customer reference, items (type of cloth, quantity), status (pending, processing, ready, delivered), total amount, and dates.
- `Pricing.js`: Stores the price list for different laundry/press services, allowing dynamic pricing calculation.

Controllers (`src/controllers/`)

These contain the actual business logic for each feature. They receive the request, process it using the Models, and send a response.

- `authController.js`: Handles user login and registration. Generates JWT tokens upon successful authentication.
- `customerController.js`: Logic to create, read, update, and delete (CRUD) customer records.
- `orderController.js`: Handles complex order logic. Creates new orders, calculates totals based on `Pricing`, updates order statuses, and triggers notifications.
- `dashboardController.js`: Aggregates data (like total revenue, active orders, recent activity) to serve the dashboard UI in a single API call for efficiency.
- `settingsController.js`: Manages application configurations, like updating the pricing list.

Routes (`src/routes/`)

These files map specific URL endpoints to their corresponding controller functions.

- `auth.js`, `customers.js`, `orders.js`, `dashboard.js`, `settings.js`: Each file defines the endpoints for its domain (e.g., `router.post('/', orderController.createOrder)`).

Middleware (`src/middleware/`)

- `auth.js`: A crucial security file. It intercepts incoming requests to protected routes, extracts the JWT from the `Authorization` header, verifies it, and attaches the user info to the request object. If the token is missing or invalid, it rejects the request before it reaches the controller.

Utilities (`src/utls/`)

Helper modules that perform specific background tasks.

- `pdfGenerator.js`: Generates a PDF bill/receipt dynamically based on order details using a PDF library.
- `whatsapp.js`: Integrates with a WhatsApp API to send automated notifications to customers when their order status changes (e.g., "Your clothes are ready for pickup").

4. Frontend Components Detailed Breakdown

Application Routing (`src/app/`)

Uses the Next.js App Router paradigm.

- `login/page.tsx`: The login screen. Unauthenticated users are redirected here.
- `(protected)/layout.tsx`: A wrapper layout for all internal pages. It ensures the user is authenticated before rendering the children and provides the structural

shell (sidebar, top bar).

- **(protected) /dashboard/, customers/, orders/, settings/**: The main pages of the application, fetching data from the backend APIs and displaying it. They map directly to the backend routes.

UI Components (src/components/)

- **AppLayout.tsx**: The main structural component containing the navigation sidebar and header. It wraps the content of protected pages.
- **ui.tsx**: A collection of reusable, stylized UI elements (buttons, inputs, cards, tables, modals) ensuring visual consistency across the app.

State Management & Context (src/context/)

- **AuthContext.tsx**: A React Context provider that holds the global authentication state (is the user logged in? who is the user?). It provides functions to login, logout, and securely store the JWT token (usually in localStorage or cookies), making this state available anywhere in the application.

API Interactions (src/lib/)

- **api.ts**: An Axios or Fetch wrapper. It acts as the single source of truth for making backend requests. It automatically attaches the JWT token from the AuthContext to every outgoing request, streamlining API calls from individual components and preventing repetitive code.